Shopify price monitor: explainer for client calls

Private notes for Siddharth. Remove `docs/private/` before the public publish (`docs/publish-checklist.md`, step 1).

How it works, in plain words

Every morning a scheduled job on GitHub reads the public product catalog of 25 US online stores. It asks for US prices, goes slowly, and follows each store's published rules. For every product option it records the price, the "was" price when there is a sale, and whether it is in stock.

Before anything is published, the run checks itself. One product per store is read a second time from a separate source on the same store, and the prices must match. The store must show US dollars. A row that breaks a rule is set aside. If the run as a whole looks wrong (too many rows missing, prices jumping by a factor of 100), it is held, and yesterday's good data stays live.

Then it compares today with yesterday and writes down every change: price up or down, sale started or ended, sold out, back in stock, new or removed. The latest prices go out as a CSV file, the full history as a monthly Parquet file, and a dashboard and README update themselves with today's numbers.

The three key design choices

1. **Politeness is a whole-run decision, not a per-store one.** Many stores share Shopify's servers. The run has one pace for everything, pauses every store when any store is asked to slow down, and stops completely when the host keeps refusing. It never spreads requests over more machines to get around a limit.
2. **Prove the data before publishing it.** Three sources (catalog feed, product feed, page data) and a currency proof on every run; a batch guard holds a run that looks wrong instead of guessing. A held run never overwrites good data.
3. **History without a database server.** The current state lives in a small CSV in git (only real changes show up as diffs); the full history is a monthly Parquet file attached to a GitHub release. DuckDB does the merge inside the run. Nothing to host, nothing to pay for.

The weakest point, and how it is covered

Shopify's rate limits on GitHub's shared runners. GitHub-hosted runners share IP addresses, and Shopify limits requests per IP. On a bad day a run gets fewer stores done. Covered by: a 38-minute request budget (the run always ends inside 45 minutes), a daily rotating store order (a cut never hits the same stores every day), stores that miss a day keeping their last good prices with a visible "not updated" mark, an alert issue when too many stores miss, and a store list sized for the budget. For a paying client the same code runs from a dedicated machine with a stable address and an agreed pace, which removes this weakness.

Real incidents (from docs/incidents.md)

- **INC-001, too fast for a shared host:** per-store pacing alone sent about 1.7 requests a second to Shopify's shared servers; stores started showing challenge pages. Fixed with one run-wide pace and a circuit breaker.
- **INC-002, missing evidence:** early test runs counted retries but not their reason, so every retry had to be treated as a rate limit. Fixed by recording the cause of every retry.
- **INC-003, first full dry run held:** free gifts ($0 prices) were counted as bad rows, and a loading bug ignored reviewed content-screen results. Both fixed, with regression tests.
- **INC-004, "slow down" obeyed by one store only:** Shopify answered with "retry in 60 seconds"; only that store waited, and the host escalated to challenges. Fixed: any "slow down" pauses the whole run.
- **INC-005, IP throttled from the start:** after many test runs in one night, the host refused almost everything. Fixed: the run stops politely, the list was sized for a 45-minute run, and test runs are rationed.

Seven questions a client may ask

1. **"How do I know the numbers are right?"** Every run checks a product from each store against a second source and proves the currency; a run that looks wrong is held, and you keep the last good data. You can see the checks of every run.
2. **"What if a site changes its layout?"** The checks catch it the same day, I get an alert, and the last good data stays live until I fix the collector, usually within a day.
3. **"Can you track my competitors?"** Yes, if the prices are public. I start with a free sample of 20-50 real rows from your exact sites so you can judge before paying.
4. **"Is this legal?"** It reads public pages only, follows each site's robots.txt, goes slowly, and collects no personal data. Site owners can ask for removal.
5. **"How fast can I get it?"** A sample in a few days; a running daily monitor usually within one to two weeks, depending on the sites.
6. **"What do I receive?"** Your choice: CSV or Excel, a Google Sheet, a database table, an API or a dashboard, on a daily, weekly or hourly schedule.
7. **"How much does it cost?"** A fixed monthly price for a running monitor, or a one-time price for a single collection, quoted after the free sample. No hourly billing.